Machine Learning–Based Network Intrusion Detection System (NIDS)

Md Shariful Islam
Computer Science Department
University of Northern British Columbia
Prince George, BC V2N 4Z9, Canada
islam5@unbc.ca

Fan Jiang
Computer Science Department
University of Northern British Columbia
Prince George, BC V2N 4Z9, Canada
fan.jiang@unbc.ca

***Abstract*—This work applies machine learning techniques to improve network intrusion detection using the CIC-IDS-2017 dataset, which includes realistic examples of both normal and attack traffic. Traditional intrusion detection systems rely on static signatures, making it difficult to recognize new or evolving attack patterns. To overcome this limitation, the dataset was preprocessed, analyzed, and optimized using Recursive Feature Elimination (RFE) to identify the most relevant flow-based features for anomaly detection. The results show that features related to average packet size, forward and backward packet length behavior, packet timing, destination ports, and total packet volume provide the highest impact on model performance. Several supervised learning algorithms were trained and compared, including Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost, CatBoost, and a Voting Classifier. The evaluation demonstrates that ensemble models consistently achieve over 99% accuracy, outperforming simpler classifiers. These findings suggest that feature-optimized machine learning approaches can significantly improve intrusion detection accuracy and support future development of real-time detection systems.**

***Index Terms*—Network Intrusion Detection, Machine Learning, Ensemble Learning, Feature Selection, CIC-IDS-2017.**

I. Introduction

Intrusion Detection Systems (IDS) play a critical role in protecting modern network infrastructures from unauthorized access, data theft, and malicious behavior. Traditional IDS solutions are mainly signature-based, which limits their ability to detect new or evolving attack patterns that do not match predefined rules. With the growth of network traffic and the increasing variety of cyberattacks, machine learning has emerged as an effective approach for identifying anomalies by learning behavioral patterns from data. The CIC-IDS-2017 dataset provides a realistic environment for evaluating intrusion detection methods, as it includes real network flows, multiple attack types, and detailed statistical features extracted from connections. By applying data pre-processing, exploratory analysis, and feature selection, machine learning models can learn meaningful distinctions between normal and malicious traffic. This study focuses on selecting the most relevant flow-based features and comparing multiple supervised learning algorithms to determine which model provides the most accurate and reliable detection of network intrusions.

II. Background and Related Work

Intrusion Detection Systems (IDS) form a core component of network defense by identifying malicious activity and abnormal behavior through continuous analysis of traffic flows [1], [7]. Traditional IDS implementations have historically relied on signature matching to detect known attacks. While effective for previously observed threats, signature-based systems cannot reliably detect zero-day attacks and typically exhibit high false-positive and false-negative rates when deployed in dynamic network environments [1], [7].

To address these limitations, anomaly-based detection models were introduced, beginning with Denning's intrusion-detection model, which formalized the use of statistical deviations from expected system behavior as an indication of compromise [1]. Large-scale evaluations such as the DARPA offline intrusion detection experiments demonstrated that anomaly-based systems have potential to detect new attack types, but they also highlighted challenges in establishing accurate behavioral baselines and achieving stable detection performance under changing traffic conditions [2].

Machine learning (ML) emerged as an effective approach to intrusion detection by learning statistical patterns directly from network data. Classical ML techniques including Decision Trees, Random Forests, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Naive Bayes, AdaBoost, and Gradient Boosting have been applied extensively to labeled intrusion datasets [3], [5], [6]. Random Forest models leverage bootstrap aggregation and feature randomness to improve generalization and reduce overfitting, achieving robust performance in heterogeneous network environments [3]. Boosting-based approaches such as AdaBoost combine weak learners to incrementally refine decision boundaries [4]. Comparative studies show that ensemble models generally outperform single classifiers because they can capture nonlinear feature interactions and reduce model variance [5], [6].

García-Teodoro et al. provided a detailed survey of anomaly-based intrusion detection, outlining challenges including redundancy and multicollinearity among flow-level features, the computational cost of real-time analysis, and difficulty achieving high accuracy across diverse attack types [7]. These findings highlight the importance of feature en-

gineering and dimensionality reduction for designing reliable IDS models. In this context, Garg et al. presented an experimental ML-based IDS workflow using a structured dataset of 25,192 flows, combining preprocessing, one-hot encoding of categorical variables, and Recursive Feature Elimination (RFE) to identify the most predictive feature subset [8]. Their evaluation compared multiple classical models and showed that ensemble classifiers, including Random Forest, Gradient Boosting, XGBoost, CatBoost, and a Voting Classifier, achieved accuracy values up to 97.72%, outperforming simpler methods such as Logistic Regression and Naive Bayes. The study further demonstrated that feature engineering can yield accurate IDS models with modest computational resources.

Although such work demonstrates the value of ML-based intrusion detection, many earlier evaluations are based on legacy benchmark datasets, most notably KDD-Cup'99 and related variants. These datasets include outdated attack categories, simplified flow structures, and unrealistic traffic distributions, limiting their relevance to modern environments [9]. To address this gap, the CIC-IDS-2017 dataset was developed with realistic network flows, reflecting modern intrusion techniques such as DDoS, brute-force login attempts, botnet activity, infiltration, and web-based attacks. The dataset contains rich flow-level statistical attributes including packet-size distributions, forward and backward packet lengths, inter-arrival timings, and traffic rate measures, making it well suited for evaluating ML approaches in contemporary intrusion detection.

The present research builds on these prior studies by applying a feature-optimized ML methodology to the CIC-IDS-2017 dataset. Exploratory data analysis (EDA) is used to inspect feature correlations and identify redundancy, followed by RFE to derive a compact set of informative flow features. Multiple classifiers from different model families are trained and compared using standard performance metrics—Accuracy, Precision, Recall, and F1-score—to determine which approaches are most effective in detecting network intrusions under realistic traffic conditions.

III. Methodology

This section describes the methodological approach used to design, train, and evaluate machine-learning models for intrusion detection using the CIC-IDS-2017 dataset. The overall methodology follows a supervised learning pipeline, including data preprocessing, exploratory data analysis, feature selection, model implementation, and performance evaluation using established metrics. The workflow design was informed by prior IDS research that demonstrates the benefits of data-driven feature selection and ensemble learning for network anomaly detection.

A. Dataset Description

This study uses the CIC-IDS-2017 dataset, developed by the Canadian Institute for Cybersecurity (CIC), which simulates realistic modern enterprise network traffic, including benign behavior and several categories of attack scenarios. Unlike traditional synthetic datasets, CIC-IDS-2017 incorporates up-to-date intrusion techniques such as DDoS, Brute Force, Botnet, Infiltration, Web Attacks, and Port Scanning, all generated from real systems and services in a controlled environment.

Each record represents a complete bidirectional network flow captured with CICFlowMeter. A flow consists of a sequence of packets exchanged between a source IP address and a destination IP address, bounded by timestamps marking the start and end of the session. For each flow, the dataset provides statistical features that summarize packet size, inter-arrival timing, byte volume, header information, TCP flags, and other behavioral characteristics. Every flow is labeled as either benign or malicious, enabling supervised learning for binary classification.

B. Data Preprocessing

Data preprocessing is a crucial step to ensure that the dataset is clean, consistent, and suitable for machine learning. Similar to the approach used in the reference work, preprocessing consists of data cleaning, encoding, scaling, and feature selection.

1) Data Cleaning: Prior to model training, all records containing missing values, NaN entries, infinite values, or invalid timestamps were removed. Duplicate network flows were eliminated to prevent bias caused by repeated samples. After filtering, the dataset still retained a diverse set of attack categories, enabling generalizable model training.

2) Label Encoding: The original dataset includes multiple intrusion categories such as DDoS, Web Attack, Brute Force, Port Scan, and others. For supervised binary classification, all attack labels were grouped into a single class and encoded as `ATTACK = 1`, while normal traffic was encoded as `BENIGN = 0`. This transformation allows training a binary classifier for intrusion detection, similar to established approaches in related literature.

3) Feature Scaling: Several numerical features exhibit different value scales, especially packet-size statistics and flow-based volume metrics. To ensure balanced contribution of all features during model training, standardization was applied using:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma} \tag{1}$$

C. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted to understand statistical relationships between flow-level features in the CIC-IDS-2017 dataset and to reduce redundancy prior to feature selection. Since CICFlowMeter generates a large number of statistical metrics for each network flow, correlation-based inspection provides a compact way to identify overlapping attributes and guide model design.

1) Correlation Analysis: The CIC-IDS-2017 dataset includes highly granular network flow statistics. To improve interpretability and reduce overfitting risk, this study focuses on 20 high-impact attributes obtained using Recursive Feature Elimination (RFE). These features capture behavioral patterns of network flows across the following dimensions:

- **Packet Size Statistics:** Average Packet Size, Packet Length Mean/Std/Variance
- **Directional Metrics:** Fwd Packet Length Max, Bwd Packet Length Max
- **Flow Timing:** Flow IAT Max
- **Traffic Volume:** Total Length of Fwd/Bwd Packets, Total Fwd Packets, Subflow Fwd Packets
- **Header-Based Indicators:** Bwd Header Length, Init_Win_bytes_backward
- **Protocol and Flag Indicators:** Destination Port, PSH Flag Count

These attributes characterize statistical patterns of malicious and benign flows without inspecting payload content, preserving privacy and enabling efficient real-time intrusion detection.

To quantify linear relationships between features, a Pearson correlation matrix was computed:

$$\rho_{xy} = \frac{\text{cov}(x, y)}{\sigma_x \cdot \sigma_y} \tag{2}$$

where $\text{cov}(x, y)$ denotes the covariance of variables x and y, and σ_x, σ_y represent their standard deviations.

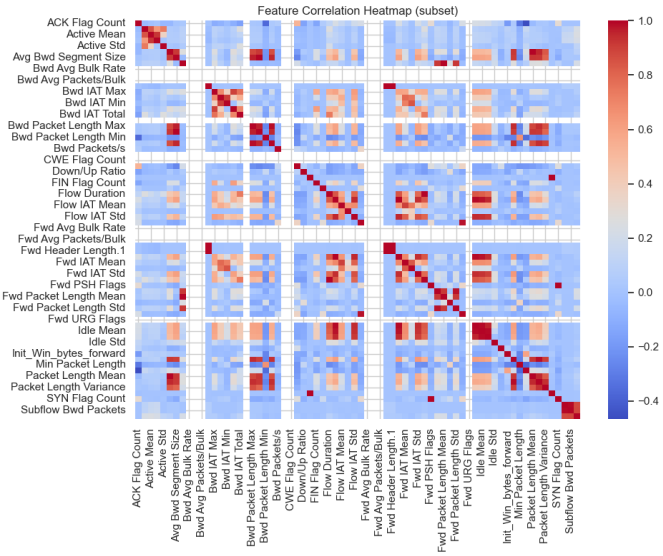

Fig. 1. Pearson correlation heatmap showing linear relationships among selected network flow features.

The heatmap in Figure 1 reveals several strong correlation clusters, such as:

- Packet Length Mean, Standard Deviation, and Variance
- Total Length of Forward Packets and Total Forward Packets
- Total Length of Backward Packets and Backward Packet Length Mean

These visual clusters indicate that multiple CICFlowMeter metrics encode the same behavioral signal using different statistical summaries.

2) Multicollinearity Analysis: The correlation structure highlights the presence of multicollinearity, where two or more features represent nearly identical information. In practice, this affects classifiers differently. Logistic Regression exhibited unstable coefficient estimates when correlated predictors were included, while KNN experienced biased distance calculations when repeated metrics implicitly weighted the same signal multiple times. Tree-based models (e.g., Random Forest, Gradient Boosting) can still achieve strong performance under collinearity, but feature importance rankings become less reliable when redundant features compete for split decisions.

To address this, features with pairwise correlation greater than 0.90 were removed, reducing the numeric feature set from 70 to 54 unique attributes. This correlation filtering prevents double-counting, reduces model variance, and lowers computational cost. Rather than eliminating features solely based on correlation threshold, the correlation map was used to inform Recursive Feature Elimination (RFE). RFE selects a representative attribute from each correlated group based on its contribution to predictive performance, ensuring that the final feature set captures core behavioral signals while avoiding redundancy.

Through data-driven selection, the resulting feature space is lower-dimensional, less noisy, and more interpretable, improving model stability during training. At the same time, it preserves the underlying patterns required for accurate intrusion detection, supporting faster convergence and better generalization to unseen traffic.

3) Recursive Feature Elimination (RFE): After correlation filtering, Recursive Feature Elimination (RFE) with a Random Forest estimator was applied to rank the remaining features and identify the top 20 attributes contributing most to classification performance. RFE removes the least informative feature iteratively, retrains the model, and repeats this process until the desired dimensionality is reached. A Random Forest classifier was selected because it captures non-linear relationships in tabular network data and provides stable importance estimates across iterations. RFE was preferred over PCA because it preserves the original feature names, which is important for intrusion detection and forensic interpretation; knowing that specific timing or packet-size indicators influenced the decision process adds practical value. The resulting 20-feature subset was used for training all models, ensuring a fair comparison across different algorithms and a compact feature space aligned with the observations from EDA.

4) Class Distribution: The CIC-IDS-2017 dataset presents an imbalanced class distribution, where benign traffic significantly outweighs attack flows. This imbalance was visualized through count plots, confirming the need for performance metrics beyond simple accuracy.

To preserve the real-world traffic proportions, no oversampling or synthetic data generation (e.g., SMOTE) was applied. Instead, the evaluation emphasizes metrics sensitive to minority-class detection, including Precision, Recall, and F1-score for the `ATTACK` class . This choice prevents artificial alterations of flow characteristics and ensures a realistic assessment of IDS performance in operational environments.

D. Train–Test Split

To evaluate the performance of the machine learning models, the dataset was partitioned into training and testing sets using stratified sampling. Stratification preserves the class distribution of benign and attack flows in both partitions, preventing biased learning toward the majority class. The split configuration was:

- **Training Set:** 70%
- **Test Set:** 30%

This approach mirrors the experimental design of comparable intrusion detection studies that benchmark performance using a 70/30 partition.

E. Feature Selection

To identify the most discriminative flow-level features, Recursive Feature Elimination (RFE) was applied using a Random Forest classifier as the base estimator. RFE iteratively removes features that contribute the least to predictive performance until the target dimensionality is reached.

The process selected 20 features that represent core characteristics of malicious network behavior, including packet-size statistics, timing variance, directional data volume, and flag-based indicators. Using a compact feature set improves computational efficiency, reduces training time, and mitigates overfitting risk, which is especially beneficial for real-time IDS deployment on resource-constrained environments.

The relative importance of the selected features is visualized in Figure 2. The importance values are derived from the mean decrease in impurity across the Random Forest trees, showing how strongly each feature influences the binary classification outcome.

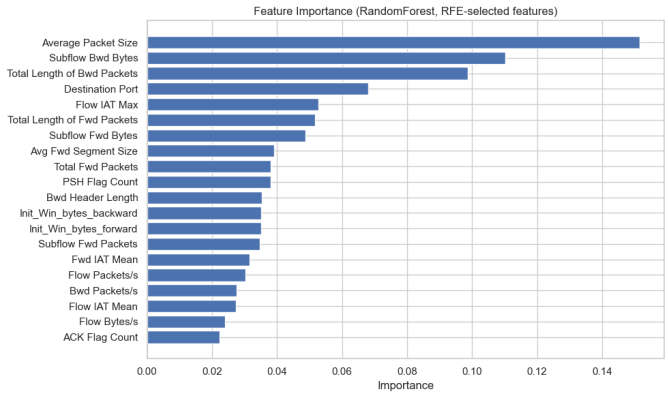

Fig. 2. Feature importance ranking of the top 20 flow-based attributes selected using RFE with Random Forest. The importance values are based on mean impurity reduction, indicating the contribution of each feature to intrusion detection accuracy.

F. Model Training and Evaluation

Multiple supervised classification models were trained to evaluate the effectiveness of the selected features and detect anomalous network behavior. The following algorithms were considered:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Gradient Boosting
- AdaBoost
- XGBoost
- CatBoost
- Voting Classifier

These models represent complementary learning strategies, enabling a comprehensive comparison across linear, distance-based, tree-based, and ensemble methods, consistent with the approach taken in related IDS research.

1) Evaluation Metrics: Model performance was evaluated using the confusion matrix, which quantifies correct and incorrect predictions:

$$\begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix} \tag{3}$$

where TP is true positives, TN true negatives, FP false positives, and FN false negatives.

From the confusion matrix, the following metrics were derived:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{5}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{7}$$

While accuracy captures overall prediction correctness, Recall and Precision are especially important in intrusion detection:

- High Recall minimizes missed attacks by reducing false negatives (FN), which is critical for defending networks.
- High Precision reduces false alarms by lowering false positives (FP), improving operational confidence and analyst trust.

Therefore, model selection prioritizes Recall and F1-Score while maintaining strong overall accuracy. This evaluation framework follows established practices in intrusion detection research and aligns with widely accepted performance benchmarking methodologies.

IV. Results and Analysis

A. Dataset Overview

1) Dataset Summary: This work uses a subset of the CIC-IDS-2017 dataset developed by the Canadian Institute for Cybersecurity (CIC), which simulates realistic enterprise network traffic including benign flows and several major attack categories. The original dataset consists of multiple daily capture files in CSV format generated by CICFlowMeter, each containing bidirectional flow records with 79 statistical attributes per flow.

This work uses four representative CSV export files from the CIC-IDS-2017 dataset: `Monday-WorkingHours.pcap_ISCX.csv`, `Wednesday-workingHours.pcap_ISCX.csv`, `Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv`, and `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`.

These files together contain 1,734,833 network flow records and 79 attributes, representing both normal activity and two high-impact attack types: distributed denial-of-service (DDoS) and port scanning. Each record corresponds to a bidirectional flow between a source and a destination IP address and includes aggregated statistical features describing packet sizes, inter-arrival times, directional byte counts, and protocol flag indicators. The flows are labeled as either benign or attack, enabling supervised learning for binary intrusion detection.

To ensure the experiments were computationally efficient while remaining representative, a 10% stratified sample was taken, resulting in 154170 records.

After preprocessing and transforming the labels into binary form (`Attack_Binary`), the class distribution was:

- **BENIGN:** 112,841 flows ($\approx 73\%$)
- **ATTACK:** 41,329 flows ($\approx 27\%$)

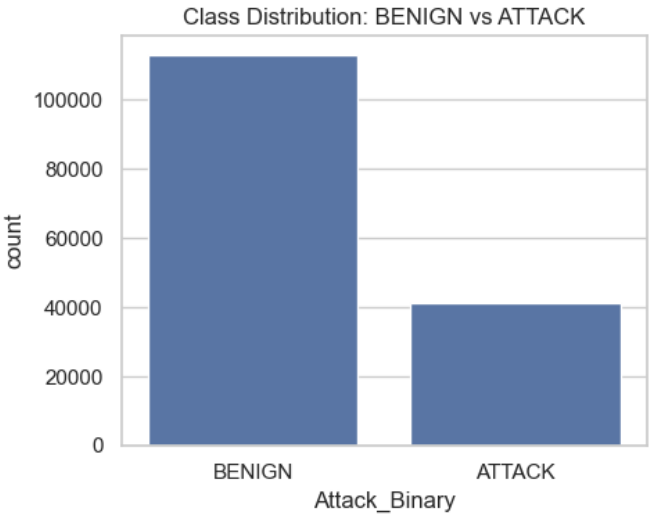

Fig. 3. Distribution of benign and attack flows in the dataset.

As Shown in Figure 3, the data is imbalanced, which is typical for network traffic datasets: most flows are benign while attacks form a smaller proportion. Consequently, model evaluation does not rely solely on accuracy. Instead, higher priority is given to precision, recall, and F1-score for the `ATTACK` class, since missing an intrusion poses a higher risk than incorrectly flagging a benign flow.

B. Performance Analysis

Table I summarizes the performance of ten supervised machine learning models evaluated on the CIC-IDS-2017 dataset using a reduced 20-feature subset selected through RFE. The results show a clear performance hierarchy, where tree-based ensemble models consistently outperform classical linear and probabilistic classifiers. The top-performing model, XGBoost, attains an accuracy of 99.94378%, closely followed by CatBoost with 99.93946%, and Random Forest with 99.93297%. These differences, although numerically subtle, are statistically meaningful due to the scale of the test set and the near-perfect operating range of all ensemble models.

TABLE I
MODEL COMPARISON RESULTS ON THE CIC-IDS-2017 TEST SET USING THE SELECTED 20-FEATURE SUBSET (VALUES SHOWN TO FIVE DECIMAL PLACES).

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
XGBoost	99.94378	99.94383	99.94378	99.94379
CatBoost	99.93946	99.93949	99.93946	99.93947
Random Forest	99.93297	99.93297	99.93297	99.93297
Decision Tree	99.90054	99.90056	99.90054	99.90055
KNN	99.72109	99.72100	99.72109	99.72103
Gradient Boosting	99.71244	99.71291	99.71244	99.71206
AdaBoost	98.98813	98.99247	98.98813	98.98957
SVM (RBF)	97.00547	97.04862	97.00547	96.96280
Logistic Regression	92.16882	92.09574	92.16882	91.99315
Naive Bayes	40.20670	79.37028	40.20670	35.68890

Figure 4 illustrates the overall accuracy across all models. It can be seen that XGBoost and CatBoost exhibit nearly identical performance, both above 99.94%, with Random Forest slightly behind at 99.93%. A single Decision Tree also achieves high performance with 99.90054%, showing that even a standalone tree structure captures significant nonlinear patterns present in network flow behavior. In contrast, SVM with an RBF kernel reaches 97.00547% accuracy, while Logistic Regression falls further to 92.16882%. Naive Bayes delivers the weakest results, with 40.20670% accuracy, confirming that its generative assumptions are incompatible with correlated flow-level features.

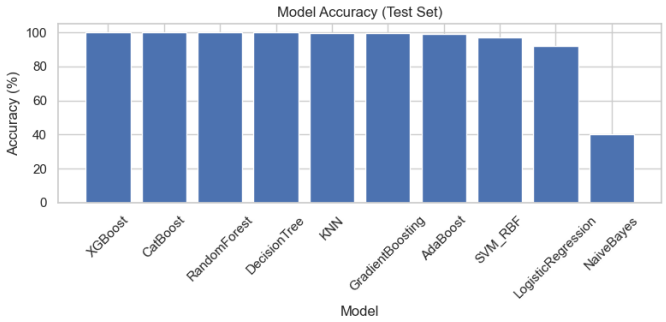

Fig. 4. Accuracy comparison bar chart for all models.

While accuracy indicates general performance, intrusion detection is sensitive to the trade-off between false positives and false negatives. Figure 5 compares accuracy and precision. The top four models (XGBoost, CatBoost, Random Forest, and Decision Tree) demonstrate nearly identical accuracy and precision values, indicating both low miss rates and low false-alarm rates. KNN and Gradient Boosting follow closely with precision values above 99.71%, benefiting from the standardized and de-correlated feature space, which enables meaningful distance metrics in KNN and stable boosting in Gradient Boosting. In contrast, Naive Bayes shows a substantial difference between accuracy (40.21%) and precision

(79.37%), where high precision arises from predicting most samples as malicious, resulting in severe misclassification of benign traffic. This validates that accuracy alone is insufficient for IDS evaluation and must be complemented by precision and recall.

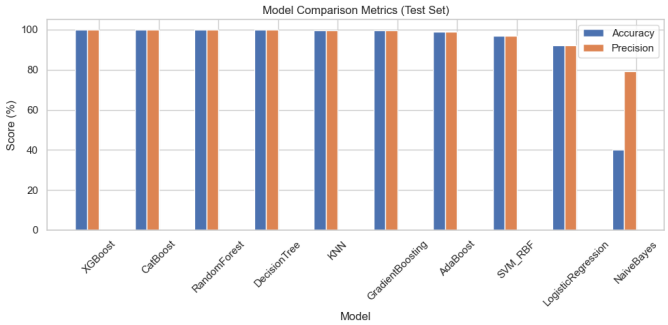

Fig. 5. Accuracy vs Precision comparison for all models.

Figure 6 compares recall and F1-score, which are critical in security applications. Recall reflects the detection rate of real attacks. XGBoost and CatBoost achieve a recall of 99.94378% and 99.93946%, respectively, meaning almost all malicious flows are detected. Random Forest and Decision Tree also demonstrate high recall above 99.90%. The F1-score, which captures the balance between precision and recall, mirrors these results, confirming that the top-performing ensembles maintain both detection sensitivity and low false-positive behavior. KNN achieves 99.72109% recall, which shows strong separation once irrelevant and correlated features are removed. Conversely, Logistic Regression falls to 91.99315% F1-score, and Naive Bayes collapses to 35.68890%, demonstrating an inability to model nonlinear interactions in flow statistics.

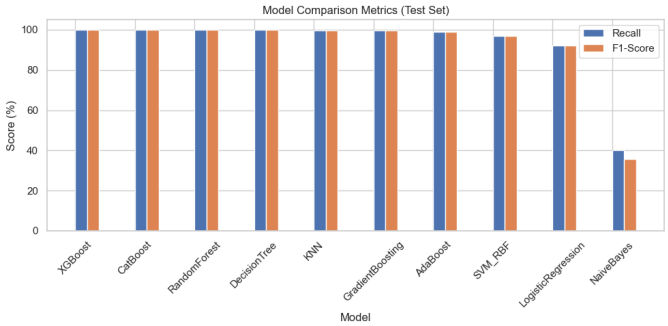

Fig. 6. Recall vs F1-Score comparison for all models.

The performance dynamics can be explained by the statistical properties of the engineered feature space. The selected 20 features represent nonlinear behavioral characteristics of network flows, including packet-size distributions, directional throughput, flow-interarrival time, window size behavior, and TCP flag patterns. Ensemble tree-based methods are well suited to exploit these interactions. Random Forest reduces overfitting by using bootstrap aggregation and random feature sampling, while XGBoost and CatBoost apply gradient boosting with strong regularization and column subsampling, enabling them to capture complex local decision boundaries that differentiate attack patterns from benign traffic. CatBoost's ordered boosting and symmetric tree structures help reduce prediction shift and improve generalization on high-dimensional flow data. Gradient Boosting also performs well but does not include advanced second-order optimization or built-in regularization, explaining its slightly lower performance.

In contrast, linear models such as Logistic Regression assume a single global linear boundary, which is inadequate for complex, multimodal attack patterns present in CIC-IDS-2017. SVM with an RBF kernel introduces nonlinearity through a radial basis transformation, but still imposes a smooth global boundary, limiting the model's ability to learn localized clusters of malicious behavior. Naive Bayes performs poorly because its assumptions of feature independence and Gaussian distributions are strongly violated: flow-level features are tightly correlated and exhibit heavy-tailed, multimodal distributions.

Overall, the experimental results show that tree-based boosting methods deliver the strongest performance (Appendix A) for flow-based intrusion detection on the CIC-IDS-2017 dataset . XGBoost achieves the best performance across all evaluation metrics, with CatBoost and Random Forest following closely. KNN and Gradient Boosting remain competitive due to the engineered feature space, while logistic and probabilistic models fail to represent the complex relationships underlying network traffic behavior. The consistency of accuracy, precision, recall, and F1-score among the top models confirms that balanced metrics, rather than accuracy alone, are essential for reliable IDS evaluation.

V. Conclusion

This work demonstrated the effectiveness of feature-optimized machine learning techniques for flow-based intrusion detection using the CIC-IDS-2017 dataset. A systematic exploratory data analysis was used to characterize correlations among network flow attributes and identify multicollinearity patterns. Based on this insight, correlation filtering and Recursive Feature Elimination (RFE) were applied to derive a compact set of 20 high-impact features representing packet-size behavior, traffic asymmetry, inter-arrival timing, and protocol indicators. This reduced feature space enables efficient learning while preserving the essential traffic characteristics that distinguish benign and malicious flows.

A comprehensive evaluation of ten classifiers showed a clear separation in performance between model families. Tree-based boosting methods—XGBoost and CatBoost—achieved the highest detection accuracy, precision, recall, and F1-score, each reaching 99.94%, followed closely by Random Forest at 99.93%. Gradient Boosting, Decision Tree, and KNN also achieved strong results, exceeding 99.7% across all metrics, illustrating the value of nonlinear decision structures in the engineered feature space. Traditional models such as SVM (RBF) and Logistic Regression exhibited lower performance

due to their reliance on smooth or linear decision boundaries, which cannot fully capture the heterogeneous behavioral patterns of modern attacks. Naive Bayes performed the

REFERENCES

[1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. 13, no. 2, pp. 222–232, 1987, doi: 10.1109/TSE.1987.232894.

[2] R. P. Lippmann, J. W. Haines, D. J. Fried *et al.*, "The 1999 DARPA off-line intrusion detection evaluation," *Computer Networks*, vol. 34, no. 4, pp. 579–595, 2000, doi: 10.1016/S1389-1286(00)00139-0.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[4] S. Elhag, A. Fernández *et al.*, "On the combination of genetic fuzzy systems and pairwise learning for improving detection rates on intrusion detection systems," *Expert Systems with Applications*, vol. 42, no. 1, pp. 193–202, 2015, doi: 10.1016/j.eswa.2014.07.047.

[5] C. F. Tsai, Y. F. Hsu *et al.*, "Intrusion detection by machine learning: A review," *Expert Systems with Applications*, vol. 36, no. 10, pp. 11994–12000, 2009, doi: 10.1016/j.eswa.2009.05.029.

[6] P. Sangkatsanee, N. Wattanapongsakorn, and C. Charnsripinyo, "Practical real-time intrusion detection using machine learning approaches," *Computer Communications*, vol. 34, no. 18, pp. 2227–2235, 2011, doi: 10.1016/j.comcom.2011.07.001.

[7] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Computers & Security*, vol. 28, no. 1–2, pp. 18–28, 2009, doi: 10.1016/j.cose.2008.08.003.

[8] I. Garg, P. Sharma, P. Sharma, V. Sharma, and G. Singh, "Network Intrusion Detection System: Machine Learning Approach," in *Proc. International Conference on Advances in Computing, Communication and Technology (ICACCTech)*, 2024, pp. 1–6.

[9] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 dataset," in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.

[10] A. Agarwal, P. Sharma, M. Alshehri *et al.*, "Classification model for accuracy and intrusion detection using machine learning approach," *PeerJ Computer Science*, vol. 7, p. 437, 2021, doi: 10.7717/PEERJ-CS.437.

Appendix A
Full Classification Reports

This appendix provides the complete classification outputs generated by scikit-learn for each model trained on the CIC-IDS-2017 dataset. The results include per-class precision, recall, F1-score, support, and overall accuracy.

```
=== Training LogisticRegression ===
              precision    recall  f1-score   support

      BENIGN    0.92674   0.96966   0.94771     33852
      ATTACK    0.90518   0.79071   0.84408     12399

    accuracy                        0.92169     46251
   macro avg    0.91596   0.88019   0.89590     46251
weighted avg    0.92096   0.92169   0.91993     46251


=== Training KNN ===
              precision    recall  f1-score   support

      BENIGN    0.99784   0.99835   0.99810     33852
      ATTACK    0.99548   0.99411   0.99479     12399

    accuracy                        0.99721     46251
   macro avg    0.99666   0.99623   0.99644     46251
weighted avg    0.99721   0.99721   0.99721     46251


=== Training SVM_RBF ===
              precision    recall  f1-score   support

      BENIGN    0.96532   0.99483   0.97985     33852
      ATTACK    0.98460   0.90241   0.94172     12399

    accuracy                        0.97005     46251
   macro avg    0.97496   0.94862   0.96078     46251
weighted avg    0.97049   0.97005   0.96963     46251


=== Training NaiveBayes ===
              precision    recall  f1-score   support

      BENIGN    0.97169   0.18856   0.31583     33852
      ATTACK    0.30777   0.98500   0.46900     12399

    accuracy                        0.40207     46251
   macro avg    0.63973   0.58678   0.39241     46251
weighted avg    0.79370   0.40207   0.35689     46251


=== Training DecisionTree ===
              precision    recall  f1-score   support

      BENIGN    0.99938   0.99926   0.99932     33852
      ATTACK    0.99798   0.99831   0.99815     12399

    accuracy                        0.99901     46251
   macro avg    0.99868   0.99878   0.99873     46251
weighted avg    0.99901   0.99901   0.99901     46251


=== Training RandomForest ===
              precision    recall  f1-score   support

      BENIGN    0.99941   0.99968   0.99954     33852
      ATTACK    0.99911   0.99839   0.99875     12399

    accuracy                        0.99933     46251
   macro avg    0.99926   0.99903   0.99915     46251
weighted avg    0.99933   0.99933   0.99933     46251


=== Training GradientBoosting ===
              precision    recall  f1-score   support

      BENIGN    0.99655   0.99953   0.99804     33852
      ATTACK    0.99870   0.99056   0.99461     12399

    accuracy                        0.99712     46251
   macro avg    0.99763   0.99505   0.99633     46251
weighted avg    0.99713   0.99712   0.99712     46251


=== Training AdaBoost ===
              precision    recall  f1-score   support

      BENIGN    0.99472   0.99143   0.99308     33852
      ATTACK    0.97682   0.98564   0.98121     12399

    accuracy                        0.98988     46251
   macro avg    0.98577   0.98854   0.98714     46251
weighted avg    0.98992   0.98988   0.98990     46251


=== Training XGBoost ===
              precision    recall  f1-score   support

      BENIGN    0.99982   0.99941   0.99962     33852
      ATTACK    0.99839   0.99952   0.99895     12399

    accuracy                        0.99944     46251
   macro avg    0.99911   0.99946   0.99928     46251
weighted avg    0.99944   0.99944   0.99944     46251


=== Training CatBoost ===
              precision    recall  f1-score   support

      BENIGN    0.99973   0.99944   0.99959     33852
      ATTACK    0.99847   0.99927   0.99887     12399

    accuracy                        0.99939     46251
   macro avg    0.99910   0.99936   0.99923     46251
weighted avg    0.99939   0.99939   0.99939     46251
```